Auditable Autonomy: Tamper-Evident, Cryptographically Verifiable Decision Provenance for Autonomous Vulnerability Remediation in Regulated Fleets

Harshavardhan Malla Independent Researcher
Email: harshavardhanmalla75@gmail.com

***Abstract*: As vulnerability remediation in large regulated fleets becomes automated, the security question shifts from *which vulnerability to patch first* to *whether the record of what was decided, and why, can be trusted after the fact*. Regulatory regimes (FISMA, FedRAMP, the FBI CJIS Security Policy) require audit trails that survive a privileged insider, yet the append-only hash chains commonly used for decision logging carry a structural blind spot: they certify internal consistency, not completeness. We make this precise with an adversarial evaluation of an automated remediation log under two adversaries and eight tampering classes (200 trials each). A naive SHA-256 hash chain detects point edits, reordering, interior deletion, and insertion against a restricted adversary, but is *blind to tail truncation and wholesale replacement* (detection rate 0.00 for both); against a realistic insider who can recompute downstream hashes it detects *nothing* (0.00 across all eight classes). We then present a verifiable provenance ledger that augments the chain with per-window Merkle commitments and periodic Ed25519-signed checkpoints anchored by an independently held latest checkpoint, in the style of a Certificate Transparency signed tree head. The ledger detects all eight attacks under both adversaries (detection rate 1.00) while adding 0.21% space overhead and verifying a 50,000-record log in under one second; append throughput remains 68,000 to 110,000 records per second. We map the construction to NIST SP 800-53 audit controls (AU-9, AU-10) and release the ledger, the adversarial harness, and the frozen results. The contribution is a deployable accountability layer for autonomous security decisions whose integrity guarantees are stated in advance and verified adversarially, not asserted.**

***Index Terms*: audit logging, tamper-evidence, hash chains, Merkle trees, digital signatures, decision provenance, vulnerability remediation, accountability, regulatory compliance, reproducibility.**

I. Introduction

Enterprise and government fleets increasingly delegate vulnerability remediation to automated pipelines: a scorer ranks (host, CVE) pairs, a capacity-constrained scheduler selects what to patch within finite windows and mandatory regulatory deadlines, and an executor applies or defers each action [1], [2]. Automation raises a question that ranking accuracy does not answer. When an auditor, an inspector general, or opposing counsel later asks *why* a specific exploited host was deferred, the answer is only as trustworthy as the log that recorded the decision, the feature values it used, and the policy version in force at the time. In regulated settings this is not optional: FISMA, FedRAMP, and the FBI CJIS Security Policy require audit records that remain attributable and intact even against insiders with administrative access [3], [4].

The common engineering answer is an append-only, hash-chained log: each record carries the cryptographic hash of its predecessor, so editing any stored record breaks the chain [5], [6]. This is necessary but not sufficient, and the gap is routinely underappreciated. A hash chain certifies that the records *present* are mutually consistent; it says nothing about records that are *absent*. An adversary who deletes the most recent k records leaves a chain that still verifies perfectly. An insider who can run the logging code can recompute every downstream hash and rewrite history into a different, internally consistent narrative. Neither attack is exotic; both are exactly the post-incident tampering that audit requirements exist to deter.

The stakes are concrete. Binding operational directives impose hard remediation deadlines for known-exploited vulnerabilities, and missing one is a reportable event [1]. When automation makes the deferral that leads to a breach, the resulting review is adversarial: the operator has both the access and the incentive to make the log support whatever account is least damaging. An audit trail that any sufficiently privileged party can silently rewrite is not evidence; it is a liability dressed as one. The security property that matters is therefore not confidentiality of the log nor even its availability, but the ability of an *independent* verifier to detect that the log has been altered or shortened, no matter who altered it. Hash chaining is widely assumed to provide this; we show it does not, and that the gap is closable cheaply.

This paper repositions a vulnerability-remediation framework around the problem that automation actually creates: *verifiable decision provenance*. Our contributions are:

- A precise, adversarial characterization of the hash-chain blind spot. Under a restricted adversary the naive chain misses tail truncation and wholesale replacement; under a realistic insider it provides *no* tamper-evidence at all (Section VI).
- A verifiable provenance ledger (Section IV) that augments the chain with per-window Merkle commitments and periodic Ed25519-signed checkpoints [7], [13], anchored by an independently held latest checkpoint as in Certificate Transparency [11], [8]. We state its guarantees in advance (Section V).

- A reproducible adversarial evaluation (Section VI): eight tampering classes, two adversaries, 200 trials each, plus runtime and space overhead at fleet scale. The ledger detects every attack under both adversaries at 0.21% space overhead and sub-second verification.
- A mapping to NIST SP 800-53 audit controls and a discussion of what the construction does and does not guarantee (Sections VII, XI).

Every number traces to a frozen artifact; the ledger and harness are released for replication.

II. Preliminaries

We use four standard primitives. *Cryptographic hash.* SHA-256 [14] maps arbitrary input to a 256-bit digest and is collision- and second-preimage-resistant: it is infeasible to find a different input with a given digest. We treat any digest collision as negligible. *Hash chain.* Linking each record to the digest of its predecessor makes the digest of the head a commitment to the entire prefix: changing any record changes the head [5], [6]. A chain proves *internal consistency* of the records present; crucially it does not commit to how many records there should be. *Merkle tree.* A binary tree whose leaves are record digests and whose internal nodes hash their children yields a single root that commits to the exact set and order of leaves, with $O(\log n)$ membership proofs [7], [8]. *Digital signature.* Ed25519 (EdDSA over Curve25519) [13] lets a holder of a private key produce signatures verifiable with the corresponding public key; without the private key, forging a signature on a new message is infeasible. Binding a signature to a Merkle root and a record count produces a commitment that a log holder cannot later repudiate or silently change, the mechanism transparency logs use to make omission detectable [11].

III. Threat Model and Requirements

System. An automated remediation pipeline emits one *decision record* per action: scoring, scheduling, approval, deferral, risk acceptance, or outcome. Each record carries the (host, CVE) pair, the maintenance window, the decision type, the feature values used, the policy (weights) version, the threat-feed snapshot versions, and the framework version. Records are written to an append-only log.

Adversary. The adversary's goal is post-hoc repudiation: to make the log tell a story other than what happened, undetectably. We consider two capability levels.

- *Restricted adversary (W).* Can read and overwrite stored log bytes but does not recompute chain hashes; models partial or forensic tampering, an append-only medium, or a tool that lacks the chain logic.
- *Insider (S).* Has log-write access and runs the logging code, so can recompute every downstream hash to restore internal consistency. This is the realistic regulated-fleet threat: a privileged operator covering tracks.

In both cases the adversary cannot forge a signature under a key it does not hold, and cannot alter an independently retained or published checkpoint. This is the standard trust anchor of transparency systems [11].

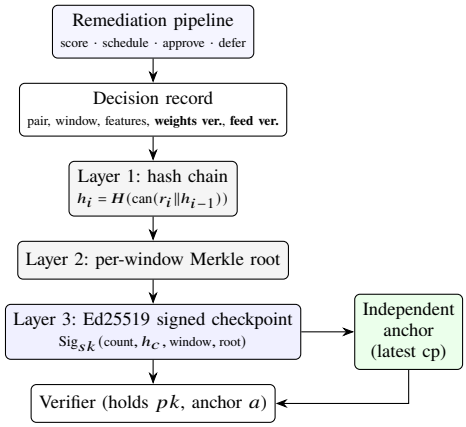

Fig. 1. The three-layer verifiable provenance ledger. Each automated decision becomes a record; Layer 1 chains it, Layer 2 commits each window to a Merkle root, and Layer 3 signs periodic checkpoints whose latest instance is held independently. The verifier checks the presented log against the signed anchor, so detection never relies on the log holder.

Alt text: A vertical block diagram. The remediation pipeline emits a decision record carrying weights and feed versions; the record feeds Layer 1 (hash chain), then Layer 2 (per-window Merkle root), then Layer 3 (Ed25519 signed checkpoint). The signed checkpoint flows both to an independent anchor store and to a verifier that also reads the anchor and the public key.

Tampering classes. We enumerate eight (Table I): field modification, reordering, interior deletion, tail truncation, forged insertion, policy (weight) rollback, threat-feed forgery, and wholesale replacement. The last two are domain-specific: silently swapping in stale weights or an old feed snapshot changes *why* a decision looks justified without changing the decision itself.

Requirements. (R1) Any of the eight tampering classes, under either adversary, must be detectable by an independent verifier. (R2) Detection must not require trusting the party that holds the log. (R3) Overhead must be compatible with fleet scale (tens of thousands of decisions per cycle).

Worked scenario. A public-facing server with a known-exploited vulnerability is deferred past its BOD 22-01 deadline because the automated scheduler exhausted its patch capacity on higher-scored hosts. The server is later compromised. During the post-incident review the operator, wishing to show the deferral was justified, has motive to (a) truncate the log so the deferral record is the last surviving entry with no contradicting outcome, or (b) rewrite the deferred record's feature values or the weights version so the decision appears to follow then-current policy. Under a naive chain both edits can be made to verify: (a) leaves an intact chain, and (b) an insider recomputes the downstream hashes. The reviewer cannot distinguish the doctored log from the truth. This is precisely the failure the ledger prevents.

IV. The Verifiable Provenance Ledger

The ledger has three layers (Fig. 1); the first is the construction already present in the remediation framework, the second and third are the contribution.

Layer 1: append-only hash chain. Each record's `record_hash` is the SHA-256 [14] of its canonical JSON

Algorithm 1 AppendRecord(r, state)

$p \leftarrow$ state.lastHash ▷ genesis 0^{64} if empty
r.prior $\leftarrow p$
r.hash $\leftarrow H(\text{can}(r))$ ▷ prior is inside can(r)
append r to log; state.lastHash $\leftarrow r$.hash
if end of window **or** checkpoint interval reached **then**
    $\rho \leftarrow$ MR(hashes of window so far)
    emit $s \leftarrow \text{Sig}_{sk}(\text{count}, r.\text{hash}, r.\text{window}, \rho)$

Algorithm 2 VerifyLedger(log, checkpoints, a, pk)

$p \leftarrow 0^{64}$
for each r in log **do** ▷ Layer 1
    if r.prior $\neq p$ **or** $H(\text{can}(r)) \neq r$.hash **then**
        return REJECT
    $p \leftarrow r$.hash
if $\neg\text{Verify}_{pk}(a)$ **then return** REJECT ▷ trusted anchor
if |log| $\neq a$.count **or** log[a.count].hash $\neq a$.head **then**
    return REJECT ▷ truncation / replacement
if MR(all log hashes) $\neq a$.root **then return** REJECT
for each checkpoint s in checkpoints **do** ▷ Layer 3
    if $\neg\text{Verify}_{pk}(s)$ **or** log[s.count].hash $\neq s$.head **then**
        return REJECT
return ACCEPT

serialization (keys sorted, tight separators, `record_hash` excluded). Each record's `prior_record_hash` is the predecessor's `record_hash`; the genesis predecessor is 64 zeros. Editing a stored record changes its recomputed hash and breaks the next link, so a restricted adversary's point edits are caught.

Layer 2: per-window Merkle commitment. Within each maintenance window the record hashes are the leaves of a binary SHA-256 Merkle tree [7]; the root commits to the exact multiset and order of records in that window. A checkpoint can therefore commit to a whole window in one hash, and membership of any record can be proven in logarithmic space.

Layer 3: signed checkpoints with an external anchor. Periodically (every window, and at the log head) the auditor signs a *checkpoint*: the tuple (record count, head hash, window id, Merkle root) under an Ed25519 private key [13]; verifiers hold only the public key. The latest checkpoint is retained independently by the auditor or published to a transparency log, exactly as a Certificate Transparency monitor retains a signed tree head [11]. Verification (i) replays the hash chain, (ii) checks every checkpoint signature, and (iii) checks that the presented log reproduces the *independently held* latest checkpoint's count, head hash, and Merkle root. Step (iii) is what defeats truncation and replacement: a short or rewritten log cannot reproduce a count and root it never signed, and the adversary cannot re-sign.

Construction. Let $r_1, \ldots, r_n$ be the records in append order, H be SHA-256, and can$(\cdot)$ the canonical serialization that excludes `record_hash`. Define $h_0 = 0^{64}$ (genesis) and, for each i, $p_i = h_{i-1}$ (the prior link) and $h_i = H(\text{can}(r_i \| p_i))$. A window w with record hashes $\{h_j\}_{j \in w}$ has Merkle root $\text{MR}(\{h_j\})$. A checkpoint at index c is $s_c = \text{Sig}_{sk}(c, h_c, w_c, \text{MR}(h_{1..c}))$. Algorithms 1 and 2 give the append and verify procedures; the verifier is parameterized by the trusted anchor a (the genuine latest checkpoint) and the public key pk.

Worked example. Consider the deferral scenario of Section III as a four-record window: r_1 scores the public-facing server's (host, CVE) pair; r_2 schedules three higher-scored hosts, exhausting capacity; r_3 defers the server with `weights_version = w-2026Q2-v1`; r_4 records the breach outcome. The window closes with a signed checkpoint $s_4 = \text{Sig}_{sk}(4, h_4, w, \rho)$, and the auditor retains s_4 as the anchor a. An insider now wants the log to end at the justified deferral, so truncates to r_1, r_2, r_3 and recomputes the chain so it verifies internally. VerifyLedger replays the three-record chain successfully (Layer 1 passes), then reaches the anchor check: $|\text{log}| = 3 \neq a.\text{count} = 4$, and the Merkle root over three records does not equal a.root. The log is rejected. Had the insider instead rewritten r_3's `weights_version` to an older value to make the deferral look policy-compliant, the leaf hash of r_3 would change, the window root would change, and the same anchor check would fail. Neither edit survives, which is precisely the accountability the naive chain failed to provide.

V. Security Analysis

We state the guarantees VerifyLedger provides to an honest auditor holding the genuine latest checkpoint a and the signer's public key pk, assuming SHA-256 collision-resistance and Ed25519 unforgeability. Throughout, "caught" means VerifyLedger returns REJECT.

Theorem 1 (Content integrity)**.** *Any modification, reordering, or interior deletion of a retained record is caught.*

Proof. Changing any field of r_i changes $H(\text{can}(r_i))$, so either the recomputed self-hash fails the Layer-1 check, or, if the adversary fixes r_i.hash and all downstream links, r_i's leaf changes and so does the Merkle root over the full log, which no longer equals a.root (step iii). Reordering changes prior-links; interior deletion makes the next record's prior-link reference a digest no longer present. Both change the root. By collision-resistance the adversary cannot find a colliding substitute, so detection is certain. □

Theorem 2 (Completeness / truncation resistance)**.** *Truncation or extension relative to the anchored count is caught, up to a residual of at most the inter-checkpoint interval.*

Proof. a binds the true count $n^\star$ and head $h_{n^\star}$ under pk. A presented log of length $n' \neq n^\star$ fails $|\text{log}| = a$.count; a log of equal length but different content fails the head or root equality. Forging a fresh a' with the adversary's count requires an Ed25519 forgery. The sole gap is records created after the last anchored checkpoint, bounded by the interval (Table III); within anchored history, loss is always caught. □

Theorem 3 (Non-repudiation of policy context)**.** *Rolling back the weights version or forging a feed-snapshot version is caught.*

TABLE I

TAMPERING CLASSES AND DETECTION RATE (200 TRIALS EACH). W: RESTRICTED ADVERSARY; S: INSIDER WHO RECOMPUTES HASHES. "CHAIN" IS THE NAIVE APPEND-ONLY HASH CHAIN; "LEDGER" IS THE SIGNED-CHECKPOINT CONSTRUCTION.

ALT TEXT: A TABLE OF EIGHT ATTACK ROWS. THE NAIVE CHAIN DETECTS SIX ATTACKS UNDER THE RESTRICTED ADVERSARY BUT SCORES 0.00 ON TAIL TRUNCATION AND WHOLESALE REPLACEMENT, AND 0.00 ON ALL EIGHT UNDER THE INSIDER. THE LEDGER SCORES 1.00 ON EVERY ATTACK UNDER BOTH ADVERSARIES.

	Chain		Ledger	
Tampering class	W	S	W	S
Modify field	1.00	0.00	1.00	1.00
Reorder	1.00	0.00	1.00	1.00
Delete (interior)	1.00	0.00	1.00	1.00
Truncate tail	**0.00**	0.00	1.00	1.00
Insert forged	1.00	0.00	1.00	1.00
Weight rollback	1.00	0.00	1.00	1.00
Feed-version forge	1.00	0.00	1.00	1.00
Wholesale replace	**0.00**	0.00	1.00	1.00

Proof. Both fields are inside can(·), so altering them is a content edit and Theorem 1 applies. This is what makes the evidence about *why* a decision was taken, not only *that* it was recorded. □

P4 (independence). Verification compares against a and pk, never against the party storing the log, satisfying R2. The only trust assumptions are that sk is held outside the log host and that a is retained independently.

Corollary 1 (Soundness)**.** *Under the assumptions of Theorems 1-3 and P4, an honest verifier accepts a presented log only if it is identical to the one the auditor signed, modulo the bounded residual of Theorem 2.*

The evaluation in Section VI confirms these guarantees empirically across all eight tampering classes and both adversaries.

VI. EVALUATION

Setup. We implement both schemes and an adversarial harness that mirrors the framework's hash-chain semantics exactly. For each of the eight tampering classes and each adversary (W, S) we run 200 independent trials on seeded logs of 400 records across 8 windows, injecting the tamper at random positions and measuring whether an independent verifier rejects the log. Overhead is measured on logs of 1,000 to 50,000 records.

Detection. Table I and Fig. 2 report the results. The naive chain behaves as its design promises only against the restricted adversary, and even then has two structural blind spots: tail truncation and wholesale replacement are undetected (0.00), because both leave an internally consistent chain. Against the insider the chain collapses entirely: detection is 0.00 for all eight classes, since an attacker who recomputes hashes faces no obstacle. The signed-checkpoint ledger detects every class under both adversaries (1.00), as Properties P1-P3 predict.

Which layer does the work. Table II attributes each detection to the mechanism that catches it. Against the restricted adversary, point tampering is caught by Layer 1 alone (broken self-hash or prior-link); against the insider, who repairs Layer 1, detection always falls to the anchored Layer-2/3 check, which is why truncation and replacement, invisible to Layer 1 entirely, are caught there. This makes precise that the contribution is not a stronger hash but an *external commitment*: the signed anchor is the single mechanism that closes both structural blind spots.

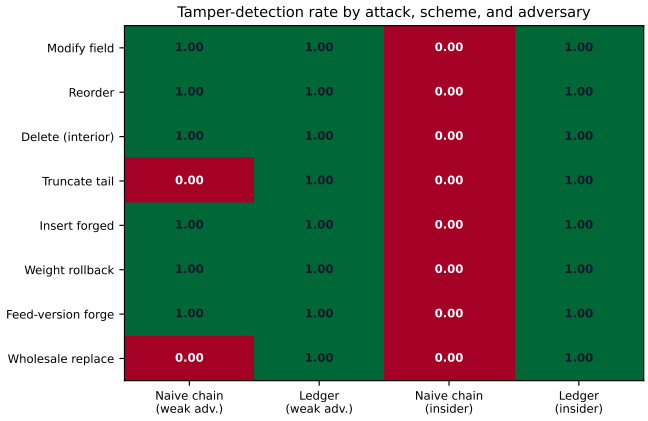

Fig. 2. Tamper-detection rate by attack, scheme, and adversary. Green is 1.00 (always caught), red is 0.00 (never caught). The naive chain (left two columns) has red cells; the ledger (right two columns) is uniformly green.

Alt text: A color heatmap, eight attack rows by four columns. The two naive-chain columns contain red 0.00 cells (all cells red in the insider column; truncate and replace red in the restricted column); the two ledger columns are entirely green at 1.00.

TABLE II

THE LEDGER MECHANISM THAT CATCHES EACH ATTACK. LAYER 1: HASH CHAIN; LAYER 2/3: MERKLE ROOT COMMITTED IN THE INDEPENDENTLY ANCHORED SIGNED CHECKPOINT.

ALT TEXT: A TABLE MAPPING THE EIGHT ATTACKS TO THE DETECTING MECHANISM. POINT EDITS AND STRUCTURAL EDITS ARE CAUGHT BY LAYER 1 AGAINST THE RESTRICTED ADVERSARY; EVERY ATTACK IS CAUGHT BY THE ANCHORED LAYER 2/3 SIGNED CHECKPOINT AGAINST THE INSIDER, INCLUDING TRUNCATION AND WHOLESALE REPLACEMENT.

Tampering class	Restricted adv.	Insider
Modify / rollback / forge	Layer 1 self-hash	Layer 2/3 anchor
Reorder / delete / insert	Layer 1 links	Layer 2/3 anchor
Truncate tail	(undetected)	Layer 3 count/anchor
Wholesale replace	(undetected)	Layer 3 signature

Overhead. Fig. 4 reports cost. Appending (hashing) sustains 68,000 to 110,000 records per second. Full verification is linear in log length: 10.5 ms at 1,000 records and 925 ms at 50,000 records, dominated by rehashing, with a small additive signature-check term. Signed checkpoints add 0.21% space, effectively constant in the log size because checkpoints are emitted per window rather than per record. The ledger therefore meets R3: a full cycle of tens of thousands of decisions is verifiable end to end in well under a second.

Where the verification cost goes. The augmented verifier does everything the naive one does (replay the chain, the dominant cost) plus a Merkle-root recomputation and a bounded number of signature checks. At 50,000 records the naive replay takes 505 ms and the full augmented verification

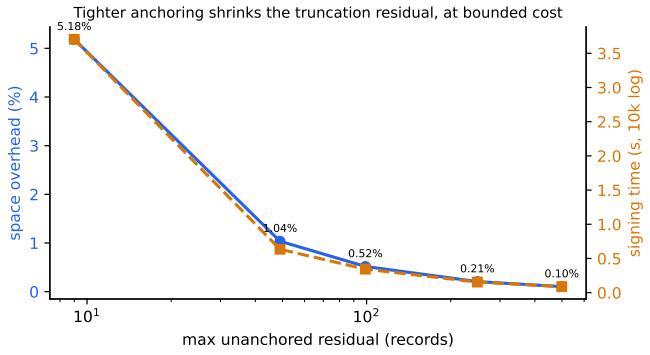

Fig. 3. The checkpoint-interval tradeoff. Tightening the anchoring interval shrinks the maximum unanchored-truncation residual (horizontal axis, log scale) at a bounded cost in space (blue) and signing time (amber, for a 10,000-record log). Even the leftmost point, a 9-record residual, costs only 5.2% space.

Alt text: A line chart. As the maximum unanchored residual decreases from about 500 to about 9 records, the space-overhead curve rises from roughly 0.1 to 5.2 percent and the signing-time curve rises from under 0.1 to about 4 seconds, showing a smooth, bounded cost for tighter anchoring.

TABLE III

CHECKPOINT-INTERVAL TRADEOFF ON A 10,000-RECORD LOG. A TIGHTER INTERVAL SHRINKS THE UNANCHORED-TRUNCATION RESIDUAL AT A MODEST, BOUNDED COST IN SIGNING TIME AND SPACE.

ALT TEXT: A TABLE WITH FIVE INTERVAL ROWS. AS THE CHECKPOINT INTERVAL DECREASES FROM 500 TO 10 RECORDS, THE MAXIMUM UNANCHORED WINDOW FALLS FROM 499 TO 9 RECORDS WHILE SPACE OVERHEAD RISES FROM 0.10 TO 5.19 PERCENT AND SIGNING TIME RISES FROM 0.08 TO 4.01 SECONDS.

Interval	Checkpoints	Residual (≤)	Sign time (s)	Space (%)
10	1000	9	4.01	5.19
50	200	49	0.75	1.04
100	100	99	0.40	0.52
250	40	249	0.15	0.21
500	20	499	0.08	0.10

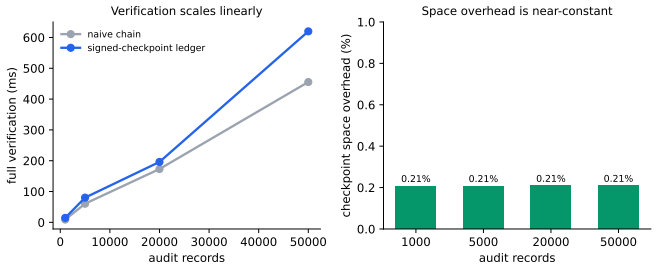

Fig. 4. Left: full verification time scales linearly for both schemes; the ledger adds a small constant for signature checks. Right: checkpoint space overhead is near-constant at 0.21%.

Alt text: Two charts. The left line chart shows verification time rising roughly linearly from about 10 ms at 1,000 records to about 925 ms at 50,000 records, with the ledger curve slightly above the naive-chain curve. The right bar chart shows checkpoint space overhead flat at about 0.21 percent across all log sizes.

925 ms, so the added integrity machinery accounts for roughly 420 ms, of which the signature checks are a small constant (200 Ed25519 verifications at a per-window interval) and the remainder is the single Merkle pass. The added cost is thus linear in log length with a small constant, and independent of how many checkpoints are present beyond the signature-check term. For logs an order of magnitude larger than a single fleet cycle, verification remains well within an interactive budget, and sealed historical segments need not be re-verified once their anchoring checkpoint is checked.

The completeness residual is a tunable knob. Property P2 leaves a residual: records created after the most recent anchored checkpoint are not yet covered, so the checkpoint interval bounds the window of undetectable recent truncation. This is a design parameter, not a flaw, and the cost of shrinking it is modest. Table III measures the tradeoff on a 10,000-record log. Anchoring every 10 records caps the unanchored window at 9 records for 5.2% space and 4.0 s of signing; anchoring every window (here, every 250-500 records) caps it at a few hundred records for 0.10 to 0.21% space and under 0.15 s of signing. An operator chooses the interval from the audit requirement: tight intervals for high-assurance criminal-justice logs, looser intervals where a bounded recent window is acceptable. Streaming anchoring (signing every record) drives the residual to zero at the highest signing cost and is the appropriate setting when even a single unanchored decision is unacceptable.

Even zero residual is affordable. The question is whether per-record signing is too slow to keep up. It is not: raw Ed25519 signing sustains 30,703 signatures per second and verification 9,935 per second on a single core in our measurements. A fleet emits on the order of thousands of decisions per maintenance window, hours apart, so signing every decision consumes a small fraction of one core's capacity. The interval is therefore a policy choice about evidence latency, not a performance constraint: operators can anchor every record when the audit requirement demands it, and relax to per-window anchoring otherwise.

Robustness and false positives. The detection results are stable across the experiment's randomization. Each cell of Table I aggregates 200 trials in which the tamper is injected at an independently drawn position, and the rates are degenerate at 0 or 1: detection does not depend on *where* the tamper lands, only on *what* structural invariant it violates, as Properties P1-P3 predict. Before each trial the clean log is verified under both schemes; across all 1,600 clean verifications (8×200) neither scheme produced a single false positive, so the ledger never rejects an untampered log. The asymmetry is the intended one: the augmented ledger adds detections without adding false alarms.

VII. Compliance Mapping

The construction discharges concrete audit controls. NIST SP 800-53 **AU-9 (Protection of Audit Information)** requires audit records be protected from unauthorized modification and deletion: P1 and P2 provide detection of both, including by privileged users [3]. **AU-10 (Non-repudiation)** is served by the Ed25519 signatures binding each checkpoint to the auditor key (P4). **AU-9(3)** (cryptographic protection) is met by SHA-256 plus EdDSA. The same evidence supports FedRAMP

audit requirements and the CJIS Security Policy's logging and accountability provisions for criminal-justice information [4]. Because verification is independent of the log holder (R2), the evidence is meaningful to an external assessor, not only to the operator.

VIII. Deployment Considerations

Integration. The ledger sits behind the remediation pipeline's existing decision-logging call: every scorer, scheduler, and executor action already emits a record, so adoption is a drop-in replacement of the log writer plus a checkpoint emitter on the window boundary. No change to the prioritization logic is required, which is why the construction is independent of the scorer.

Key management. The signing key is the trust root and must live outside the host that holds the log, for example in a hardware security module or a separate auditor service; otherwise an insider who holds both defeats P4. Verifiers need only the public key and the latest anchor. Rotation is handled by recording the key identifier in each checkpoint and publishing rotation events, standard EdDSA practice [13]. Key compromise is out of scope and composes with HSM and threshold-signing defenses.

Performance budget. A maintenance cycle of 5×10^4 decisions appends in under a second (Fig. 4) and is fully verifiable in under a second; checkpoint signing at a per-window interval adds tens of milliseconds. These costs are negligible against the hours-to-days cadence of patch windows, so the ledger imposes no operational drag. Storage grows linearly with decisions plus the constant-fraction checkpoint overhead, and old segments can be sealed by their anchoring checkpoint and moved to cold storage without losing verifiability.

IX. Related Work

Tamper-evident logging. Hash-linked records for tamper-evident timestamping originate with Haber and Stornetta [5]. Schneier and Kelsey applied chaining, with per-entry forward-secure keys, to audit logs on untrusted machines [6]; Bellare and Yee formalized forward integrity for the same setting [9], and Ma and Tsudik later gave forward-secure sequential aggregate signatures that compress an entire log's authentication into constant state [10]. These schemes strengthen *content* integrity and, with forward security, limit the damage of key compromise, but a pure chain still does not commit to log *length*, which is the gap we target. Crosby and Wallach showed that naive chains are costly to audit and introduced history trees for efficient tamper-evident logging [8]; our per-window Merkle commitment is in this lineage and reduces per-window verification to a single root.

Transparency systems. Certificate Transparency [11] contributes the operational pattern we adopt: a signed commitment to log state, held and gossiped independently, that makes omission and equivocation detectable rather than merely modification. Key-transparency systems such as CONIKS extend the idea to per-user directories with privacy [12]. Our anchor is the same signed-tree-head mechanism, specialized to a remediation decision stream and to an offline auditor who retains the latest checkpoint rather than a gossip network.

TABLE IV

Where the verifiable provenance ledger sits. Content: detects edits; Completeness: detects truncation/omission; Insider: holds against an adversary who recomputes hashes; Context: binds policy/feed version into the evidence.

Alt text: A comparison table. Plain hash chains provide content integrity only. Schneier-Kelsey and history trees add efficiency. Certificate Transparency adds completeness and insider resistance via external anchoring. This work provides all four properties including decision-context binding.

Approach	Content	Complete	Insider	Context
Plain hash chain [5]	✓			
Secure audit log [6]	✓			
History tree [8]	✓	✓		
Cert. Transparency [11]	✓	✓	✓	
This work	✓	✓	✓	✓

Accountability for automated decisions and our position. Table IV situates the construction: prior tamper-evident logging gives content integrity and, with an external monitor, completeness, but is not specialized to decision provenance and does not bind the policy and threat-feed context that explains *why* an automated action was taken. Our contribution is to instantiate these guarantees for *autonomous remediation*, to characterize the hash-chain blind spot adversarially in that setting under two explicit adversaries, to bind policy and feed versions into the evidence, and to quantify the cost of closing the gap at fleet scale. This is orthogonal to work on the prioritization decision itself (for example EPSS-based ranking [15]): we make the *record* of any such decision trustworthy, whatever scorer, heuristic, or learned model produced it.

X. Discussion

From "which to patch" to "can we prove what we did." Most vulnerability-management research optimizes the ranking decision [15]. As that decision is automated, a second, orthogonal question becomes load-bearing: whether the institution can later *prove*, to an adversarial reviewer, what its automation decided and on what basis. Verifiable provenance answers it independently of the scorer, so the same ledger serves any prioritization method, including future machine-learning ones whose decisions are harder to explain and therefore more in need of an incorruptible record.

What pre-stated, adversarially-tested guarantees buy. A common failure mode in security engineering is to assert that a hash chain "makes the log tamper-proof." Our evaluation shows that claim is false in two concrete ways against a realistic insider. Stating the guarantees as checkable properties (P1-P4) and then attacking the system across eight classes converts a vague assurance into a measured one. This discipline, rather than any single primitive, is what makes the construction defensible to an auditor or in a dispute.

Generality. Nothing in the ledger is specific to vulnerability data; it applies to any append-only stream of automated decisions that must remain attributable, such as access-control changes or model-driven enforcement actions. The

vulnerability-remediation setting supplies a concrete, regulated, high-stakes instance with explicit deadline and accountability requirements (BOD 22-01, CJIS), which is why we evaluate it there.

XI. Limitations

The completeness guarantee (P2) holds only up to the most recent anchored checkpoint; decisions made after the last checkpoint and before the next are not yet covered, so the inter-checkpoint interval bounds the undetectable truncation window. We checkpoint per window to keep this small, but streaming anchoring (signing every record, or near-real-time publication) would remove it at higher signing cost. The evaluation uses synthetic logs with the production record schema; it exercises the integrity machinery faithfully but does not model operational failure modes such as key compromise or clock manipulation, which are out of scope and would compose with standard key-management and trusted-time defenses. Finally, the trust anchor must genuinely be independent; if the adversary controls both the log and the retained checkpoint, no log-only scheme can help.

XII. Conclusion

Automating remediation moves the trust boundary from the decision to its record. We showed adversarially that the append-only hash chains used for that record have a structural blind spot, undetectable to a restricted adversary for truncation and replacement and useless against a realistic insider, and we closed it with a verifiable provenance ledger combining per-window Merkle commitments and independently anchored Ed25519 checkpoints. The ledger detects all eight tampering classes under both adversaries at 0.21% space overhead and sub-second verification for 50,000 records. The result is an accountability layer for autonomous security operations whose guarantees are pre-stated, adversarially tested, and reproducible.

Data Availability Statement

The data and code that support the findings of this study are openly available in the research-papers repository at https://github.com/Harshavardhanmalla-Labs/research-papers/tree/main/paper1. The ledger implementation, the adversarial harness, and the frozen result tables (`detection.csv`, `overhead.csv`, and `provenance_summary.json`) are included and regenerate every reported number deterministically from the fixed seeds.

Disclosure Statement

No potential conflict of interest was reported by the author.

Funding

No funding was received.

References

[1] CISA, "Binding operational directive 22-01: Reducing the significant risk of known exploited vulnerabilities," U.S. Cybersecurity and Infrastructure Security Agency, Nov. 2021. [Online]. Available: https://www.cisa.gov/news-events/directives/bod-22-01

[2] NIST, "Guide to enterprise patch management planning: Preventive maintenance for technology," National Institute of Standards and Technology, Special Publication 800-40 Rev. 4, Apr. 2022.

[3] NIST, "Security and privacy controls for information systems and organizations," National Institute of Standards and Technology, Special Publication 800-53 Rev. 5, Sep. 2020.

[4] Federal Bureau of Investigation, "Criminal justice information services (CJIS) security policy, version 5.9.2," 2022. [Online]. Available: https://www.fbi.gov/file-repository/cjis-security-policy-v5-9-2-20221207.pdf

[5] S. Haber and W. S. Stornetta, "How to time-stamp a digital document," *Journal of Cryptology*, vol. 3, no. 2, pp. 99-111, 1991.

[6] B. Schneier and J. Kelsey, "Secure audit logs to support computer forensics," *ACM Trans. Information and System Security*, vol. 2, no. 2, pp. 159-176, 1999.

[7] R. C. Merkle, "Protocols for public key cryptosystems," in *Proc. IEEE Symp. Security and Privacy*, 1980, pp. 122-134.

[8] S. A. Crosby and D. S. Wallach, "Efficient data structures for tamper-evident logging," in *Proc. USENIX Security Symp.*, 2009, pp. 317-334.

[9] M. Bellare and B. S. Yee, "Forward integrity for secure audit logs," Computer Science and Engineering Dept., University of California, San Diego, Tech. Rep., Nov. 1997.

[10] D. Ma and G. Tsudik, "A new approach to secure logging," *ACM Trans. Storage*, vol. 5, no. 1, pp. 1-21, 2009.

[11] B. Laurie, A. Langley, and E. Kasper, "Certificate transparency," RFC 6962, Internet Engineering Task Force, Jun. 2013.

[12] M. S. Melara, A. Blankstein, J. Bonneau, E. W. Felten, and M. J. Freedman, "CONIKS: Bringing key transparency to end users," in *Proc. USENIX Security Symp.*, 2015, pp. 383-398.

[13] S. Josefsson and I. Liusvaara, "Edwards-curve digital signature algorithm (EdDSA)," RFC 8032, Internet Engineering Task Force, Jan. 2017.

[14] NIST, "Secure hash standard (SHS)," National Institute of Standards and Technology, FIPS PUB 180-4, Aug. 2015.

[15] J. Jacobs, S. Romanosky, B. Edwards, I. Adjerid, and M. Roytman, "Exploit prediction scoring system (EPSS)," *Digital Threats: Research and Practice*, vol. 2, no. 3, pp. 1-17, 2021.